

Reporte de Herramientas de Ingeniería (Atributo HI)

Radar Acústico Monoestático para Estimación de Distancia

David García Cruz, Saddy Guzmán Rodríguez
 Área Académica de Ingeniería en Computadores
 Instituto Tecnológico de Costa Rica
 CE1110 – Análisis de Señales Mixtas

Correo: d.garcia.1@estudiantec.cr, sadguzman@estudiantec.cr

I. SELECCIÓN DE TÉCNICAS, RECURSOS, HERRAMIENTAS O MÉTODOS

Para resolver el problema del radar acústico monoestático con limitaciones de tiempo real y hardware embebido de bajo costo, se seleccionaron los siguientes recursos:

- 1) **Microcontrolador ESP32-WROOM-32D:** Seleccionado por su arquitectura de doble núcleo Xtensa a 240 MHz, periférico DAC integrado de 8 bits para síntesis de señales continuas arbitrarias y ADC de 12 bits con atenuación programable (11 dB). Su capacidad de memoria SRAM (320 KB) permite almacenar vectores complejos de 512 puntos sin desbordamiento.
- 2) **Entorno PlatformIO y VS Code:** Empleado para la gestión de compilación cruzada con optimización de nivel -O3, administración del mapa de memoria flash y subida de firmware a alta velocidad (921600 baudios).
- 3) **Ecosistema Científico Python (NumPy, SciPy, Matplotlib):** Utilizado para modelar analíticamente el canal de propagación, simular reflectores múltiples y verificar la equivalencia numérica entre la correlación temporal y espectral.
- 4) **Telemetría y Visualización en Tiempo Real:** Se desarrollaron scripts en Python (`diagnostico_radar.py` y `osciloscopio.py`) comunicados por puerto serie a 115200 baudios utilizando tramas estructuradas en formato JSON.
- 5) **Herramientas Asistidas por Inteligencia Artificial:** Utilizadas como soporte para la aceleración en la estructuración modular de código C++, validación de ecuaciones del factor de giro de Cooley-Tukey y refinamiento de la documentación técnica.

II. APLICACIÓN DE TÉCNICAS, RECURSOS Y MÉTODOS EN EL PROBLEMA

La aplicación metodológica se ejecutó en tres fases complementarias:

A. Fase 1: Simulación y Modelado Matemático

Se generaron ondas sintéticas en Python y se implementó la FFT Radix-2 de Cooley-Tukey por decimación en el tiempo.

Se validó que el cálculo de correlación espectral:

$$R_{rs} = \text{IFFT}\left(\text{FFT}(r) \cdot \text{conj}(\text{FFT}(s))\right) \quad (1)$$

con $N_{\text{FFT}} = 512$ puntos de *zero-padding* fuera idéntico a la correlación directa temporal con un error residual inferior a 10^{-14} . Se demostró que la modulación por chirp (1.0 kHz a 3.0 kHz con ventana Hann) eliminó los lóbulos secundarios falsos producidos por tonos armónicos simples.

B. Fase 2: Implementación DSP en C++ Embebido

Se estructuró una librería modular (`dsp_radar.h` y `dsp_radar.cpp`) con buffers estáticos para prevenir la fragmentación del montículo (*heap*). Se implementó un filtro IIR Biquad pasa banda de segundo orden centrado en 2.0 kHz para eliminar el ruido electromagnético de alimentación y componentes fuera de banda.

C. Fase 3: Instrumentación y Diagnóstico Gráfico

Mediante el script `diagnostico_radar.py`, el microcontrolador transmite ráfagas completas de 256 muestras capturadas y 225 puntos de correlación, permitiendo inspeccionar en pantalla la señal cruda, la señal filtrada, la zona ciega y la posición exacta del pico de correlación.

III. ADAPTACIÓN DE TÉCNICAS Y MÉTODOS AL SISTEMA FÍSICO

El paso de la teoría al hardware real requirió adaptaciones de ingeniería:

- 1) **Compensación Dinámica de la Tasa de Muestreo (f_s):** La latencia interna de `analogRead()` redujo la frecuencia real a ~ 9400 Hz. Se adaptó el algoritmo midiendo con `micros()` la duración real de cada captura para computar la distancia con la frecuencia instantánea exacta.
- 2) **Directividad Física y Atenuación de Multitrayectoria:** El micrófono electret omnidireccional captaba reflexiones de paredes laterales a ~ 74 cm. Se adaptó una bocina cónica directiva que concentró la captación en el eje frontal.

3) **Calibración de Cero (OFFSET_CALIBRACION_CM):**

El cono acústico y el retardo de grupo de los filtros analógicos y digitales introducían un sesgo constante de +10.0 cm. Se adaptó el firmware restando este retardo intrínseco de instrumentación, logrando precisión de ± 1.1 cm.

4) **Filtro de Mediana Móvil:** Para suprimir saltos aislados provocados por ruido transitorio, se implementó un filtro de mediana de 5 muestras que estabiliza la visualización en el monitor serie.